LAB11

Data Structure

Lab11

- The deadline for lab11 submission is 28th May at 11:59 pm.
- If you have any questions, please contact TA through email. (justin043435@hanyang.ac.kr)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 11 scores will only be accepted during next classes.

- Code name: {student_id}_p11.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p11	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab 11. Topological Sorting

- Design and implement topological sorting.
- During Topological sort, when multiple candidate vertices exist, enqueue the one with lower index in vertex array first!
- Vertex and edge information will be given in the input file.
- The program should print the results of topological sort.

Lab 11. Topological Sorting

- The input file will follow this structure:
  - X *// Number of vertices*

    v1 v2 v3 … vX *// Vertex values (unique integers)*

    v1 v3 *// Following lines : Directed edges (v1 → v3)*

    v2 vX

    vX v1

    …

Lab 11. Topological Sorting

- Structures

```c
typedef struct _Queue{
    int size;
    int *key;
    int front;
    int rear;
    int count;
} Queue;

typedef struct _Graph{
    int size;
    int *vertex;
    int **edge;
} Graph;
```

- Functions

```c
Queue *CreateQueue(int X);
void Enqueue(Queue *Q, int item);
int Dequeue(Queue *Q);
Graph *CreateGraph(int X);
void InsertEdge(Graph *G, int u, int v);
void Topsort(Graph *G);
```

Lab 11. Topological Sorting

- **Queue *CreateQueue(int X)**
  - Creates a new queue with the size of X.
- **void Enqueue(Queue *Q, int item)**
  - Inserts a new element to the rear of the queue.
- **int Dequeue(Queue *Q)**
  - Removes and returns the front element of the queue
- **Graph *CreateGraph(int X)**
  - Creates vertices and adjacency matrix.
- **void InsertEdge(Graph *G, int u, int v)**
  - Adds a directed edges from u to v.
- **void Topsort(Graph *G)**
  - Performs a topological sort and print the order.

Lab 11. Topological Sorting

```c
#include<stdio.h>
#include<stdlib.h>

typedef struct _Queue{
    int size;
    int *key;
    int front;
    int rear;
    int count;
} Queue;

typedef struct _Graph{
    int size;
    int *vertex;
    int **edge;
} Graph;

Queue *CreateQueue(int X);
void Enqueue(Queue *Q, int item);
int Dequeue(Queue *Q);
Graph *CreateGraph(int X);
void InsertEdge(Graph *G, int u, int v);
void Topsort(Graph *G);
```

```c
int main(int argc, char *argv[]){
    FILE *fi = fopen(argv[1], "r");
    int X, u, v;

    fscanf(fi, "%d", &X);

    Graph *G = CreateGraph(X);

    for(int i = 0; i < X; i++){
        fscanf(fi, "%d", &G->vertex[i]);
    }

    while(fscanf(fi, "%d %d", &u, &v) != EOF){
        InsertEdge(G, u, v);
    }

    printf("Topological Sort: ");
    Topsort(G);

}
```

Lab 11. Topological Sorting

Graph *CreateGraph(int X)

void InsertEdge(Graph *G, int u, int v)

```c
typedef struct _Graph{
    int size;
    int *vertex;
    int **edge;
} Graph;
```

- Input Example

```
5
5 4 1 2 3
5 3
4 2
3 1
```

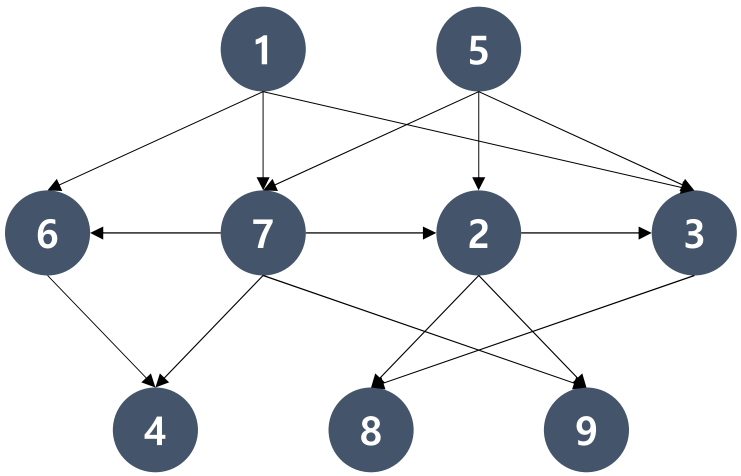
Vertices

5	4	1	2	3

Adjacency Matrix

5	0	0	0	0	1
4	0	0	0	1	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	1	0	0

Lab11. Example of Topological sorting



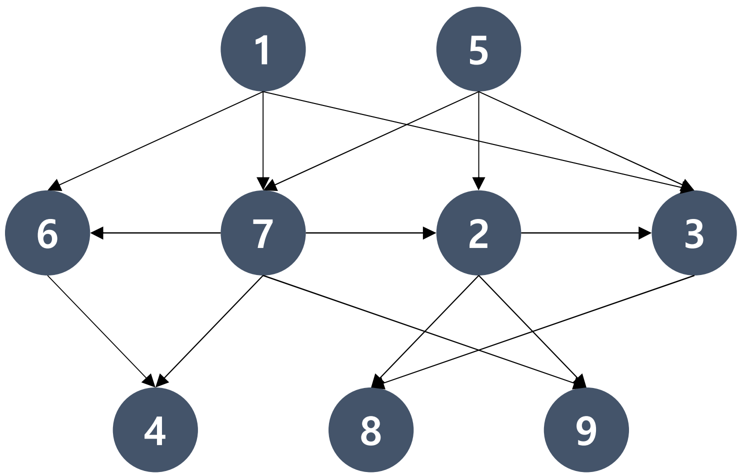
Vertices

1	5	6	7	2	3	4	8	9

Adjacency Matrix

1		0	1	1	0	1	0	0	0
5	0		0	1	1	1	0	0	0
6	0	0		0	0	0	1	0	0
7	0	0	1		1	0	1	0	1
2	0	0	0	0		1	0	1	1
3	0	0	0	0	0		0	1	0
4	0	0	0	0	0	0		0	0
8	0	0	0	0	0	0	0		0
9	0	0	0	0	0	0	0	0	

Lab11. Example of Topological sorting



Vertices

1	5	6	7	2	3	4	8	9

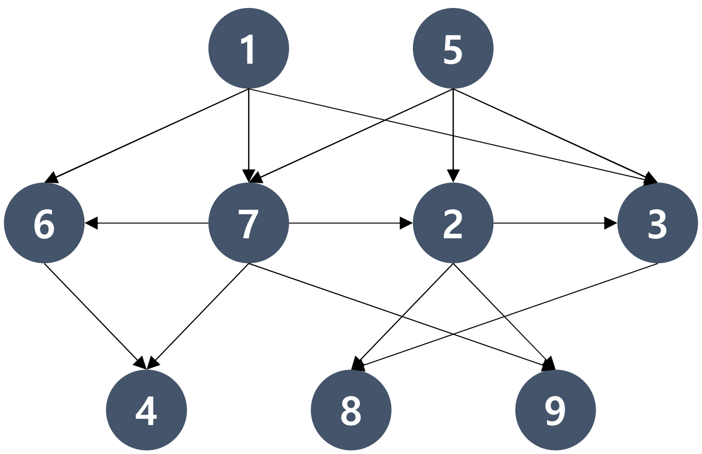
Adjacency Matrix

1		0	1	1	0	1	0	0	0
5	0		0	1	1	1	0	0	0
6	0	0		0	0	0	1	0	0
7	0	0	1		1	0	1	0	1
2	0	0	0	0		1	0	1	1
3	0	0	0	0	0		0	1	0
4	0	0	0	0	0	0		0	0
8	0	0	0	0	0	0	0		0
9	0	0	0	0	0	0	0	0	

Indegree

0	0	2	2	2	3	2	2	2

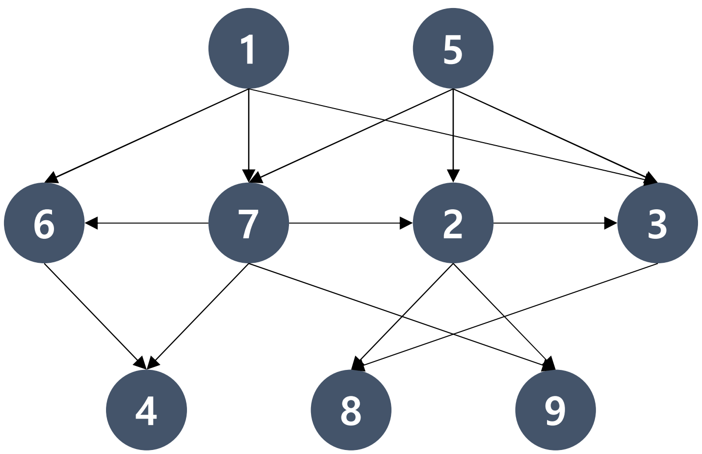
Lab11. Example of Topological sorting



1	5	6	7	2	3	4	8	9	Vertices
0	0	2	2	2	3	2	2	2	Indegree

Q

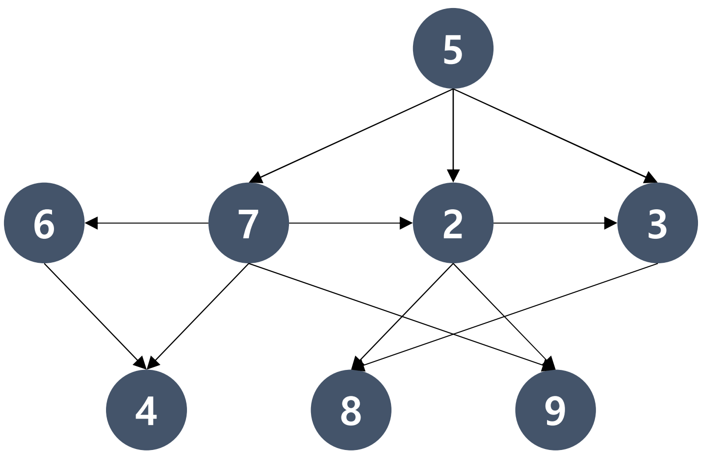
Lab11. Example of Topological sorting



1	5	6	7	2	3	4	8	9	Vertices
0	0	2	2	2	3	2	2	2	Indegree

Q
1 5

Lab11. Example of Topological sorting

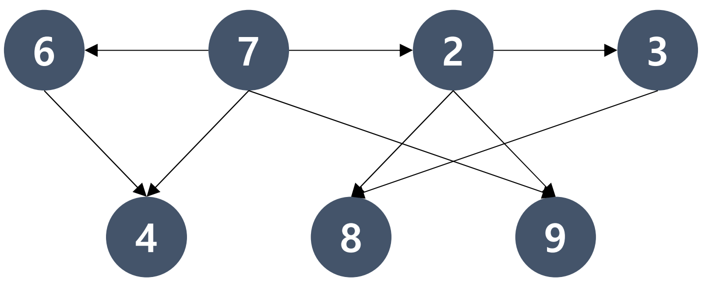


1	5	6	7	2	3	4	8	9	Vertices
0	0	1	1	2	2	2	2	2	Indegree

Q: **5**

1

Lab11. Example of Topological sorting

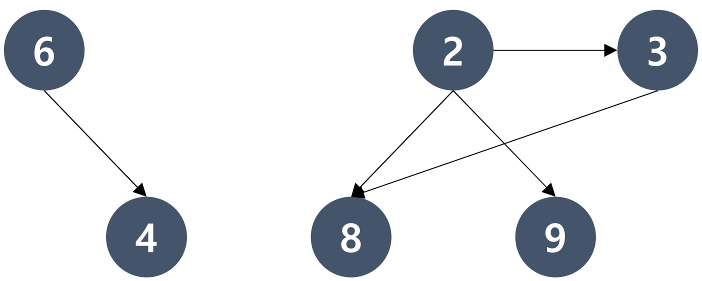


1	5	6	7	2	3	4	8	9	Vertices
0	0	1	0	1	1	2	2	2	Indegree

7 Q

1 5

Lab11. Example of Topological sorting

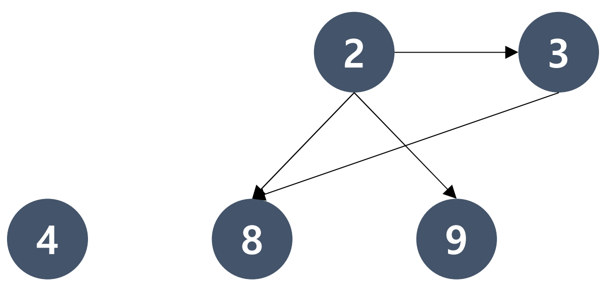


1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	1	1	2	1	Indegree

6 2 Q

1 5 7

Lab11. Example of Topological sorting

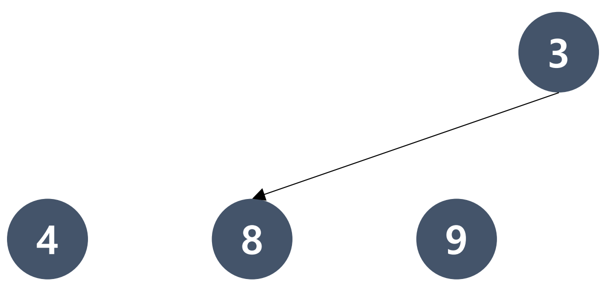


1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	1	0	2	1	Indegree

2 4 Q

1 5 7 6

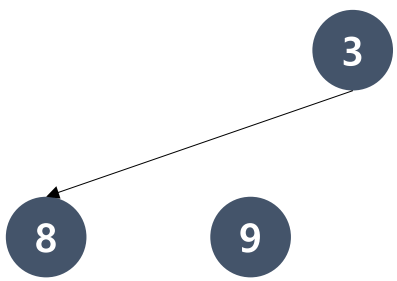
Lab11. Example of Topological sorting



Vertices	1	5	6	7	2	3	4	8	9
Indegree	0	0	0	0	0	0	0	1	0

Q	4 3 9
	1 5 7 6 2

Lab11. Example of Topological sorting



1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	0	0	1	0	Indegree

Q: **3 9**

1 5 7 6 2 4

Lab11. Example of Topological sorting

1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	0	0	0	0	Indegree

9 8	Q

1 5 7 6 2 4 3

Lab11. Example of Topological sorting

1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	0	0	0	0	Indegree

8	Q

1 5 7 6 2 4 3 9

Lab11. Example of Topological sorting

1	5	6	7	2	3	4	8	9	Vertices
0	0	0	0	0	0	0	0	0	Indegree

Q

1 5 7 6 2 4 3 9 8

Lab 11. Topological Sorting

- **Example 1)**

Input

```
5
4 2 3 1 5
4 2
2 3
3 1
1 5
```

Output

```
Topological Sort: 4 2 3 1 5
```

Lab 11. Topological Sorting

- **Example 2)**

Input

```
6
30 40 50 60 10 20
30 40
30 50
40 60
50 60
60 10
10 20
```

Output

```
Topological Sort: 30 40 50 60 10 20
```

Lab 11. Topological Sorting

- **Example 3)**

Input

```
9
1 7 4 9 8 3 2 5 6
1 7
1 4
7 9
4 9
7 8
4 3
9 2
8 2
3 5
2 6
5 6
```

Output

```
Topological Sort: 1 7 4 8 9 3 2 5 6
```